CUNEF UNIVERSIDAD

Relational databases · Databases practice

ER concepts: summary

1. **Basics**: Entity · Attributes and their kinds · Keys: candidate, primary, composite
2. **Relationships**: Relationships and degree · Type of correspondence: 1:1, 1:N, M:N · Look-across cardinality and participation · Ternary and higher-degree relationships
3. **Weak entities and constraints**: Weak entities · Exclusive, inclusive and inclusion relationships
4. **Extended ER model**: Generalization and specialization · Categories (union types) · Aggregation
5. **Building the model**: The temporal dimension · Steps for building the model

Basics

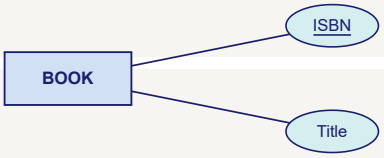
Entity

A real or abstract object of interest to the organization, about which we can — and want to — obtain information.

- Rule i: it must have its **own existence**.
- Rule ii: each occurrence must be **distinguishable** from the others.
- Rule iii: all occurrences must have the **same types of attributes**.
- Name entity types with a singular noun (BOOK, not BOOKS).

> **Example.** In a library, BOOK is an entity type, "Don Quixote" is one of its occurrences, and Title is one of its attributes.

> **Common mistake.** Promoting a single property (a phone number, a colour) to an entity. If it has no existence of its own and nothing to describe beyond its value, it is an attribute.



The entity type BOOK (rectangle) with a key attribute ISBN (underlined) and a descriptor attribute Title.

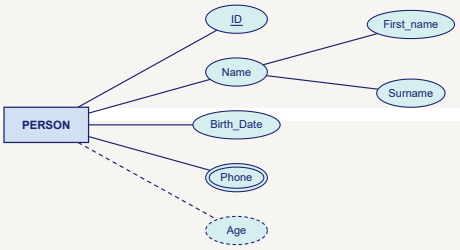
Attributes and their kinds

A property or characteristic of an entity, common to all its occurrences: a basic unit of information that serves to identify or describe it.

- **Simple** (indivisible) vs **composite** (Address = street, number, city, province, postal code).
- **Single-valued** (one value per occurrence) vs **multivalued** (several phone numbers).
- **Mandatory**: must take at least one value for every occurrence.
- **Derived**: calculable from other attributes (Age from Birth_Date) — redundant, so remove it; keep it only for efficiency.

> **Example.** PERSON(ID, Name, Birth_Date, Phone, Age): ID identifies, Name is composite (first name + surnames), Phone is multivalued and Age is derived.

> **Common mistake.** Storing a derived attribute as if it were required. The course says derived attributes must be removed from the schema and may be kept only as an optional efficiency measure.



Key ID (underlined), composite Name with its components, simple Birth_Date, multivalued Phone (double ellipse) and derived Age (dashed).

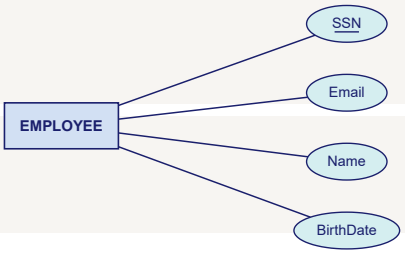
Keys: candidate, primary, composite

A primary key is an attribute, or a minimal set of attributes, that gives unique access to each occurrence of an entity.

- Candidate keys: every attribute set that can identify an occurrence.
- Primary key: the candidate you choose.
- Composite key: several attributes that identify together, and only together.
- Descriptor: characterizes but does not identify.

Example. EMPLOYEE(SSN, Email, Name, BirthDate): SSN and Email are both candidate keys; choosing SSN makes it the primary key and leaves Email as an alternate key.

Common mistake. Adding extra attributes to a key "to be safe". SSN + Name is not a key, because it is not minimal: SSN alone already identifies the employee.



SSN was chosen as the primary key (underlined); Email is an alternate key; Name and BirthDate are descriptors.

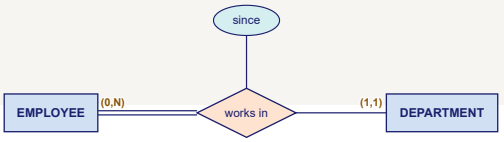
Relationships

Relationships and degree

An association between entities, governed by restrictions that determine which entities may participate in it.

- **Unary** (recursive): an entity relates to itself — an employee manages other employees.
- **Binary**: entities related two by two — EMPLOYEE works in DEPARTMENT.
- **Ternary**: three entity types — SUPPLIER supplies PART to PROJECT.
- Relationship attributes (a `since` date) belong to the diamond, not to either entity.

Common mistake. Counting occurrences instead of entity types when giving the degree. "Manages" links many employees, but only one entity type, so it is unary.



A binary relationship "works in" with its own attribute "since". Each employee works in exactly one department; a department has zero or more employees.

Type of correspondence: 1:1, 1:N, M:N

The maximum number of occurrences of one entity that can be associated with one occurrence of the other entity through a relationship.

- (1,1) with (0,N) ⇒ 1:N.
- (0,N) with (0,N) ⇒ M:N.
- (0,1) or (1,1) at both ends ⇒ 1:1.
- The mins do not change the ratio; they give participation.

Example. STUDENT (0,N) — enrols in — COURSE (0,N): both maxima are N, so the correspondence is M:N.

Common mistake. Writing "1:N" or "M:N" next to the diamond of a binary relationship. The course diagrams carry only the (min,max) tuples; the ratio is derived from them.



Both tuples have max = N, so this relationship is many-to-many (M:N). Both mins are 0, so both participations are partial.

Look-across cardinality and participation

The (min,max) at entity E states how many occurrences of E can be related to ONE occurrence of the other entity.

- max = 1 at E ⇒ E is the "one" side; max = N ⇒ the "many" side.
- min = 1 at E ⇒ every opposite occurrence relates to at least one E.
- The min = 1 and the double line sit at **opposite ends**.
- Ratio from the two maxima: (1,1)/(0,N) ⇒ 1:N.

Example. TruckDriver (1,1) — Delivers — Package (0,N): each Package is delivered by exactly one TruckDriver; each TruckDriver delivers zero or more Packages. Package participates totally (double line), TruckDriver partially.

Common mistake. Reading the tuple "look-here": taking (1,1) at TruckDriver to mean "each driver delivers exactly one package". It means the opposite: each package has exactly one driver.



The (1,1) at TruckDriver makes Package total (double line); the (0,N) at Package leaves TruckDriver partial. Maxima 1 and N ⇒ 1:N.

Ternary and higher-degree relationships

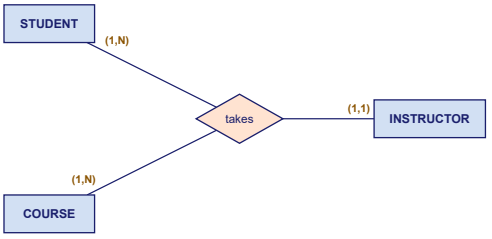
One diamond, three (or more) entities: each occurrence is a single fact that needs all of them at once.

- The maxima give the ratio: **1:1:1**, **1:1:N**, **1:M:N** or **M:N:P**.
- A max of **1** at an end means the other entities determine it: in M:N:1, the pair (student, course) is the key of the relationship.
- Taken two by two, the entities are **M:N**. A rule about a pair ("each course has one instructor") is a different, explicit constraint.
- In the logical model it becomes **one table** with a FK to each entity (see ER → Logical, "Ternary relationship").

Takes, three real facts. Split into pairs and joined again, it gains a fourth, **spurious** row: Mike · Physics · Song.

student	course	instructor
Mike	Physics	Jones
Mike	Chemistry	Song
Anne	Physics	Song

Watch the video on the website, in the ER concepts section.



A ternary relationship. Read each end with the other two fixed: for one student and course, exactly one instructor (1,1); for one course and instructor, one or more students (1,N).

Example. STUDENT (1,N) — takes — COURSE (1,N) — INSTRUCTOR (1,1): for one student and one course there is exactly one instructor, so the ratio is M:N:1.

Common mistake. Replacing the ternary with three binary relationships. Joining the pairs back invents facts: in the table above, Mike–Physics, Physics–Song and Mike–Song all exist as pairs, so the join adds "Mike takes Physics with Song", which never happened.

Weak entities and constraints

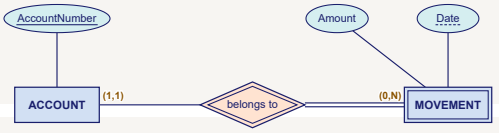
Weak entities

An entity that depends on another (its owner) for its existence; drawn as a double rectangle.

- Existence dependency: depends on the owner but has its own sufficient key.
- Identification dependency always implies existence dependency — not the other way round.
- The course contract models a double rectangle only when **both** dependencies hold.
- Weak entities in M:N relationships are highly unusual; they sit on the "many" side of a 1:N.

Example. A bank MOVEMENT has no key of its own; it is identified by its ACCOUNT number plus its date, so PK = AccountNumber + Date.

Common mistake. Marking an entity weak just because it "belongs to" another. If it has its own natural key (an invoice with a unique invoice number), it is strong.



MOVEMENT (double rectangle) is identified through the identifying relationship "belongs to" (double diamond): PK = AccountNumber + Date (partial key).

Exclusive, inclusive and inclusion relationships

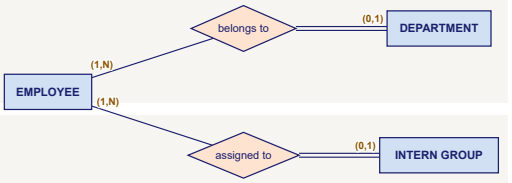
Constraints between two relationship types R1 and R2 with respect to the same entity E1.

- **Exclusive**: E1 relates either with E2 (through R1) or with E3 (through R2), never both at the same time.

- **Inclusive**: to take part in R2, E1 must previously have taken part in R1 a **certain number of times** (at least two courses before working as a product designer).
- **Inclusion**: to take part in R2, E1 must previously have taken part in R1 **once** (a marriage before a divorce).

Example. An employee is either a permanent member of staff who belongs to a department, or an intern assigned to an internship group who belongs to no department — an exclusive relationship.

Common mistake. Confusing inclusive with inclusion: the difference is whether the previous participation is required a certain number of times (inclusive) or once (inclusion).



EMPLOYEE takes part either in "belongs to" or in "assigned to", not both (exclusive). The exclusivity itself is a constraint between the two relationships, noted beside the diagram.

Extended ER model

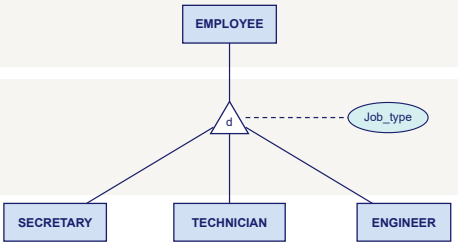
Generalization and specialization

A hierarchy connects a supertype to its subtypes through an inverted triangle; the subtypes inherit the supertype's attributes and relationships.

- Supertype on top, triangle base parallel to it, subtypes below.
- `d` disjoint · `o` overlapping · `U` union (category).
- Discriminant: a circle with a dotted edge to the triangle.
- Total vs partial: must every supertype occurrence belong to a subtype?

Example. EMPLOYEE specialised into SECRETARY, TECHNICIAN and ENGINEER, disjoint, discriminated by Job_type.

Common mistake. Creating subtypes that add nothing. A subtype is justified by its own attributes or relationships; otherwise a descriptor attribute is enough.



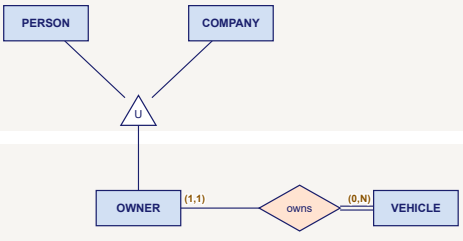
A disjoint (d) specialization of EMPLOYEE; the discriminating attribute Job_type is joined to the triangle by a dotted edge.

Categories (union types)

A category is a subtype that results from the union of several entity types: several supertypes, one subtype.

Example. Given PERSON and COMPANY, if a relationship with VEHICLE is needed, OWNER is created as a union subtype of the two: a vehicle is owned by an OWNER, who is either a person or a company.

Common mistake. Drawing OWNER as a supertype of PERSON and COMPANY. That would mean every person and company is an owner; the category says every owner is a person or a company.



OWNER is the union (U) of PERSON and COMPANY, so a single relationship "owns" can link it to VEHICLE.

Aggregation

Builds a new entity type from other entity types together with their relationship, so it can be handled at a higher level of abstraction.

Example. COMPANY and JOB APPLICANT are related through INTERVIEW, but each interview must correspond to a particular JOB OFFER. Aggregate COMPANY + INTERVIEW + APPLICANT into one entity type and relate that to JOB OFFER.

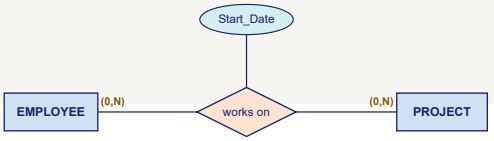
> **Common mistake.** Drawing a line straight from one diamond to another. A relationship cannot connect to a relationship; aggregation is the construct that makes it legal.

Building the model

The temporal dimension

Decide whether the data must be seen over time, and capture the time milestones with date attributes.

> **Example.** An employee can work on the same project in several periods. A Start_Date on "works on" records each period — but the pair (employee, project) alone no longer identifies an occurrence.



A date attribute on an M:N relationship records when each occurrence happened; check that each occurrence stays uniquely accessible.

Steps for building the model

The course's five steps for creating the extended E/R model, from a brief to a documented diagram.

- 1. Identify the **entities** within the system.
- 2. Determine the **keys** (identifiers) of the entities.
- 3. Establish the **relationships** between the entities, describing their degree.
- 4. **Draw** the conceptual data model.
- 5. Identify each entity's **attributes** and describe them in the data dictionary.

> **Common mistake.** Starting from tables and foreign keys. That is the logical model (the How); the conceptual model comes first and is independent of it.